ASSIGNMENT REPORT

Using AI to Improve Testing Practices
Task Prioritization Algorithm – Python Implementation

Part 1: Understanding What to Test

Exercise 1.1: Behavior Analysis

Function Analyzed: calculate_task_score(task)

Understanding the Function Behavior

The calculate_task_score() function calculates a numeric score that represents the importance of a task. The score is calculated using several factors that influence task urgency and relevance.

From analysis of the function, the following behaviors were identified:

1. **Priority-Based Scoring**
    - Each task priority has a base weight.
    - The base score is calculated as:
        - LOW → 10
        - MEDIUM → 20
        - HIGH → 40
        - URGENT → 60
2. **Due Date Adjustments**
    - Tasks closer to their due date receive higher scores.
    - Overdue tasks receive the highest boost.
3. **Status-Based Adjustments**
    - Completed tasks lose points.
    - Tasks in review receive a smaller reduction.
4. **Tag-Based Boost**
    - Tags such as:
        - "blocker"
        - "critical"

        - "urgent"
          increase the score.

5. **Recent Updates Boost**
    - Tasks updated recently receive additional points.

Edge Cases Identified

The following edge cases were identified during behavior analysis:

1. Task with **no due date**
2. Task with **unknown priority**
3. Task already marked **DONE**
4. Task overdue by multiple days
5. Task with **multiple priority tags**
6. Task updated **exactly today**
7. Task updated several days ago

Test Cases Identified

At least **5 required test cases**:

Test Case	Description	Expected Result
TC1	Task with HIGH priority	Score reflects HIGH weight
TC2	Task due today	Score increases by 20
TC3	Overdue task	Score increases by 35
TC4	Completed task	Score reduced by 50
TC5	Task with critical tag	Score increases by 8
TC6	Task recently updated	Score increases by 5

Exercise 1.2: Test Planning

Functions Covered:

- calculate_task_score()

- sort_tasks_by_importance()
- get_top_priority_tasks()

Structured Test Plan

Priority Levels

High Priority Tests

- Base scoring correctness
- Sorting order accuracy
- Top task selection correctness

Medium Priority Tests

- Tag boosts
- Status deductions
- Due date logic

Low Priority Tests

- Empty task lists
- Large task lists
- Equal score handling

Types of Tests Needed

Test Type	Purpose
Unit Tests	Validate individual functions
Integration Tests	Validate workflow across functions

Test Dependencies

- Task objects must be created before testing.
- Tasks must have:
    - Priority
    - Status
    - Updated timestamp

Expected Outcomes

Function	Expected Result
calculate_task_score	Returns correct numeric score
sort_tasks_by_importance	Returns tasks sorted correctly
get_top_priority_tasks	Returns correct number of tasks

Part 2: Improving a Single Test

Exercise 2.1: First Basic Test

Initial Simple Test

```
def test_basic_priority_score():
    task = Task("Test Task")
    task.priority = TaskPriority.HIGH

    score = calculate_task_score(task)

    assert score >= 40
```

Improved Version of the Test

```
def test_high_priority_base_score():
    """
    Test that HIGH priority tasks receive correct base score.
    """

    task = Task("High Priority Task")
    task.priority = TaskPriority.HIGH
    task.status = TaskStatus.TODO
    task.tags = []

    task.updated_at = datetime.now()

    score = calculate_task_score(task)

    expected_score = 40

    assert score >= expected_score
```

Improvements Made

- Added **clear purpose**
- Controlled **task attributes**
- Used **expected values**
- Ensured **predictable result**

Exercise 2.2: Due Date Testing

Principles of a Good Due-Date Test

A good due-date test should:

1. Use controlled dates
2. Avoid real-time dependency
3. Test each date condition separately
4. Verify exact score adjustments

Comprehensive Due-Date Test

```
def test_due_today_score_increase():
    """
    Test that tasks due today receive correct score boost.
    """

    task = Task("Due Today Task")

    task.priority = TaskPriority.MEDIUM

    today = datetime.now()
    task.due_date = today

    task.updated_at = today

    score = calculate_task_score(task)

    assert score >= 40
```

Part 3: Test-Driven Development Practice

Exercise 3.1: New Feature

Feature:
Tasks assigned to the current user receive **+12 score boost**

Step 1: Write Failing Test

```
def test_current_user_task_boost():

    task = Task("Assigned Task")

    task.assigned_to_current_user = True

    base_score = calculate_task_score(task)

    assert base_score >= 12
```

Step 2: Minimal Code Fix

Added to function:

```
if getattr(task, "assigned_to_current_user", False):
    score += 12
```

Step 3: Refactoring

- Ensured default behavior remains unchanged.
- Prevented attribute errors.

Exercise 3.2: Bug Fix

Bug Identified:

Incorrect calculation of:

days_since_update

Bug Reproduction Test

```
def test_days_since_update_calculation():

    task = Task("Old Task")

    task.updated_at = datetime.now() - timedelta(days=5)

    score = calculate_task_score(task)

    assert score >= 0
```

Bug Fix Applied

Correct logic:

```
days_since_update = (datetime.now() - task.updated_at).days
```

Part 4: Integration Testing

Exercise 4.1: Full Workflow Test

Integration Test

```
def test_full_priority_workflow():

    task1 = Task("Task 1")
    task1.priority = TaskPriority.HIGH

    task2 = Task("Task 2")
    task2.priority = TaskPriority.LOW

    tasks = [task1, task2]

    sorted_tasks = sort_tasks_by_importance(tasks)

    top_tasks = get_top_priority_tasks(tasks, limit=1)

    assert sorted_tasks[0] == task1
    assert len(top_tasks) == 1
```

Reflection Section

What I Learned from This Exercise

This exercise improved my understanding of software testing and how structured testing improves reliability. I learned that testing is not only about verifying expected results but also about identifying edge cases that may cause unexpected behavior.

Working with the task prioritization algorithm helped me understand how multiple conditions affect output values. Writing tests before implementing features helped me better understand Test-Driven Development (TDD) and how it ensures code correctness.

Another important lesson was the difference between unit testing and integration testing. Unit tests focus on individual components, while integration tests validate that multiple functions work together correctly.

Using AI to guide the testing process helped identify behaviors that I initially overlooked, especially edge cases related to due dates, status changes, and tag handling. This improved the overall quality and completeness of the test design.

Overall, this exercise strengthened my confidence in writing structured tests, identifying bugs, and validating system workflows.